

OWASP HoneyNet Experiment

by

Hannah Fitzgerald, Ethan Kortman, Michael Dedvukaj, Ashley Bekondo

Department of Electrical and Computer Engineering and Computer
Science
Tagliatela College of Engineering
University of New Haven

ABSTRACT

While universities such as the University of New Haven have a substantial security infrastructure, there is always room for improvement. Our team is looking to find any vulnerabilities the network may have to recommend security improvements for the university through using a honeynet to mirror the University of New Haven networks. This is done through having the honeypots behind the University's Connecticut Institute of Technology (CIT) network behind a firewall to protect the integrity of the University's network. The use of honeypots and honeynets are used to track attack patterns of attackers to help find vulnerabilities so that appropriate changes can be made. This project will allow the university to further their security posture as a whole in their network environment.

	3
ABSTRACT	2
Chapter I	4
INTRODUCTION	4
Introduction	4
Problem Statement	5
Project Objectives	5
Limitations of the Project	6
Definition of Terms	6
Chapter II	7
BACKGROUND AND REVIEW OF LITERATURE	7
Introduction	7
Background Related to the Project	7
Literature Related to the Project	8
Honeynet Design and Architecture	8
History of Honeynets	9
Honeynet Implementation	11
Honeynet Data Collection and Analysis	12
Honeynet Deception Strategies	15
Legal and Ethical Considerations	17
Chapter III	19
METHODOLOGY	19
Introduction	19
Design of the Project	19
Tools and Techniques Used	20
Chapter IV	21
DISCUSSION AND RESULTS	21
Introduction	21
Results and Discussion	22
Chapter V	23
CONCLUSION AND FUTURE WORK	23
Conclusion	23
Future work	24
Appendix	25
REFERENCES	28

Chapter I

INTRODUCTION

Introduction

In a world where cyber attacks and threats to security are higher than ever, it is important for companies, businesses, and individuals to protect their data in the best way that they can. This is inclusive of the University of New Haven that has highly confidential information including, but not limited to, student's personal identifying information, staff and faculty personal identifying information, and donor information. Any leaks to this information can cause great financial loss, identity theft, and difficulties maintaining the systems within the university. The capstone team has implemented multiple honeypots that highlight different systems in order to mirror the infrastructure of the University's networks, in order to recommend security changes and study what can be improved upon within the network. This is done through having the honeypots behind the University's CIT network behind a firewall to protect the integrity of the University's network. Honeypots trap attackers by posing as decoy vulnerabilities to attract prying eyes. This helps get the attackers away from the actual networks as they are distracted by the decoy vulnerability that they want to attack. The use of honeypots and honeynets are used to track attack patterns of attackers to help find vulnerabilities so that appropriate changes can be made.

Problem Statement

While universities such as the University of New Haven have a substantial security infrastructure, there is always room for improvement. Our team is looking to find any vulnerabilities the network may have to recommend security improvements for the university through using a honeynet to mirror the University of New Haven networks. Our project will address these issues by mimicking CIT's production network to increase the overall security posture of the University of New Haven.

Project Objectives

1. Implement the honeynet to track attack patterns
2. Successfully monitor traffic on the honeynet.
3. Study attack patterns and methodology to understand how attackers move through the system.
4. Have a comprehensive list of recommendations for the University to take to improve the network.

Limitations of the Project

The limitations of this project include the inability to work on the University's actual network. This is due to the fact that the network would be vulnerable to different threats and susceptible to attacks. Therefore, to protect our data as a University, we are operating behind the CIT network to mirror the network infrastructure. With more time and funding, we would be able to launch honeypots that are able to be attacked from all around the world.

Definition of Terms

Honeypot - a decoy that is intentionally made to attract attackers and malicious software.

Honeynet - a group of Honeypots scattered throughout the network infrastructure.

OWASP - Open Worldwide Application Security Project

CIT Network - The University of New Haven's Lab Network hardware.

Chapter II

BACKGROUND AND REVIEW OF LITERATURE

Introduction

The field of Cybersecurity has launched itself into one of the largest industries in the entire world. However, Cybersecurity faces thousands of intelligence and security threats daily. Cybersecurity professionals have implemented multiple tools and techniques to combat various cyber attacks. One tool that has been implemented is a Honeypot. Honeypots are a security tool that relies on vulnerabilities to help track malicious users' activity and intentions. A honeynet is the term used to describe a group of honeypots implemented on a secure network. In this specific project, OWASP Honeynets are being deployed. The Open Web Application Security Project Honeynet, or OWASP Honeynet, is implemented using accessible Web Applications to track any attempted or successful attacks.

Background Related to the Project

This project has been developed with the concept of the University of New Haven's web application security in mind. The purpose of implementing a project such as the OWASP Honeynet is to protect the students, staff, faculty, and other stakeholders of the University from any vulnerabilities in the web applications. The consequences of an attack on the University can be disastrous financially, socially, and academically. By implementing an OWASP Honeynet

system on the CIT network, the University could implement a system to ensure that those who use these web-applications daily are protected along with their information.

Literature Related to the Project

Honeynet Design and Architecture

Design and Architecture refers to the foundational aspect behind the building and deployment of a honeynet system. The design of a honeynet prioritizes isolation to prevent attackers from accessing production systems, transparency to appear authentic, and scalability to adapt to various attack scenarios. With this in mind, a honeynet system is deemed one of the better security options in detecting cyber attacks. Being that legitimate users do not interact with the honeynet, the false detection rate within the system remains low (Javadpour, et al, 2024).

The primary goal of a honeynet is for it to be attacked. This allows organizations with confidential data to better monitor and mitigate attacks and study attack patterns. When looking at the composition of a Honeynet, one must look at the individual honeypot. The main features or components of Honeypots are network spoofing function, port redirection, alerting, data capture, analysis, and control (Huang & others, 2021). Network spoofing is all about making the honeypots within the honeynet look similar, if not identical, to the applications within the real network. Port direction allows for the simulation of a working system without actually offering any legitimate services to attackers. The alerting, data capture/analysis, and control components make administrators aware of any intrusion, while simultaneously logging all generated data to allow for analysis. In this project, these individual honeypots represent different web applications that may be on the CIT's network. According to the different interaction frequency between

Honeypot and an attacker, it can be divided into high-interaction Honeypot and low-interaction Honeypot. Additionally, Honeypots can be further divided into production Honeypot and research Honeypot depending on the ultimate purpose of the deployment (Omar & others, 2024). In the case of this project, a high interaction Honeynet for a production environment would best reflect our goals.

History of Honeynets

The idea of honeynets comes from something called honeypots. Honeypots first appeared in the early 1990s. Essentially, a honeypot is a fake computer or system set up to attract hackers. The point is to watch how hackers try to break in and learn from their actions. Lance Spitzner is a key figure in this area. He helped a lot in explaining and developing honeypots. His 2002 book, *Honeypots: Tracking Hackers*, describes how honeypots work and why they are useful (Spitzner, 2002). At first, honeypots were simple; they were single systems, isolated from real networks, designed to trap attackers.

But researchers quickly saw that a single computer was not enough to study the full scope of hacker attacks. Many attackers don't just target one machine; they try to move inside networks and attack multiple systems. To get better information, experts created honeynets. These are collections of fake computers (honeypots) connected in a network that looks real. By grouping multiple fake machines together, honeynets can mimic the structure of an actual company's network. This setup allows researchers to see how attackers behave in more complex environments, including how they move between systems.

A big moment for honeynets came in 1999 with the start of the Honeynet Project. This was a group of cybersecurity researchers and experts, including Spitzner, working to build better honeynet systems. The goal was to capture hacker activity and share knowledge among the

security community. The Honeynet Project helped turn honeynets from small research experiments into practical tools that could be used in real cybersecurity defense (Honeynet Project, 2025).

Since then, honeynets have improved a lot. One major advance is virtualization technology. Virtualization allows many virtual computers to run on a single physical machine. This makes it easier and cheaper to deploy large numbers of fake systems in a honeynet. Now, honeynets can imitate whole enterprise networks with many machines, services, and different types of traffic. This setup catches a wider range of hacker tactics, from the first scans of a network to attempts to gain control over systems and move laterally within the network (Provos & Holz, 2004).

Honeynets help cybersecurity teams learn more about threats and collect important data that improves security tools. For example, they contribute to creating better intrusion detection systems that try to spot attacks early. Governments, companies, and researchers all use honeynets to understand attackers better and improve defenses. However, honeynets come with risks. A poorly controlled honeynet may be used by attackers themselves as a base to attack other targets. Or hackers could use it as a stepping stone to break into real networks. To stop this, you need strict controls like firewall rules and constant monitoring. Good honeynet design isolates the fake network from real systems to avoid such risks.

Over the past thirty years, honeynets have moved from simple traps on just one machine, to complex, automated systems that help track and analyze hackers on multiple levels. The major milestones include the original honeypot concept in the early 1990s, formation of the Honeynet Project in 1999, and later improvements in the 2010s with virtualization and cloud technology. The use of virtual machines made it easier and cheaper to run these fake systems. Researchers

could simulate big networks using software, without needing lots of physical computers. A paper from 2004 explains how virtual honeynets let people study things like botnets using safer and more flexible setups (Provos & Holz, 2004). Now, honeynets are a standard research and defense tool in cybersecurity.

Honeynet Implementation

A honeynet is a network of honeypots designed to mimic production systems, tricking attackers into interacting with fake targets so you can study their behavior. A honeypot is a single fake system or service that looks legitimate, such as a web server, database, or SSH login. A honeywall is the perimeter gateway between a honeynet, a possible production network, and the internet. Its three main goals are: data capture, data control, and automated alerting (Lackner, 2021). Setting up a honeypot starts with choosing whether you want a low-interaction honeypot, which only mimics basic services, or a high-interaction honeypot, which runs full operating systems or applications. Low-interaction setups are safer but offer limited information, while high-interaction ones provide detailed data at the cost of higher risk. Honeypots are usually deployed on virtual machines for easy isolation, recovery, and scalability. Proper segmentation ensures that even if attackers compromise the honeypot, they can't reach your real systems.

Adding deliberate vulnerabilities can encourage attackers to interact with the honeypot, while monitoring tools like Zeek, Suricata, or Snort keep track of their activity. The Honeynet controls the attacker's activity by using a honeywall gateway allowing inbound traffic to the victim systems and controlling the outbound traffic using intrusion prevention technologies (Kaur et al., 2012). These tools log network traffic, system changes, and malicious attempts,

allowing you to analyze the collected data later. Logs are typically stored on a separate server or system to prevent attackers from tampering with them.

A honeynet takes this idea further by combining multiple honeypots into a single, interconnected network that mimics a real-world environment. For example, a honeynet might include fake web servers, databases, IoT devices, and file shares. You can use tools to manage multiple honeypots, centralize logs, and coordinate activity monitoring. Simulating normal network traffic, such as users accessing files or web pages, helps make the honeynet look realistic to attackers.

Honeynets require monitoring systems like Wireshark to detect suspicious activity and alert you to potential threats. These setups allow you to study how attackers move through a network, what tools they use, and what they target. This data is valuable for improving security by identifying vulnerabilities and attack trends. Honeypots and honeynets are great tools for learning about threats without putting your real systems at risk, as long as they are set up and monitored correctly.

Honeynet Data Collection and Analysis

Data collection and analysis refers to the process of gathering data and examining it further for certain trends or patterns within the actual data. When it comes to Honeypots, the analysis and collection of data is key information for a Cybersecurity team, as it can provide insight to the attackers and the patterns of their attacks. It is vital that data is collected accurately and insightfully to fully grasp the tasks that need to be completed in order to run a successful honeypot or honeynet system onto a network. However, data collection can pose a problem for a network security team, as it can be a privacy violation.

Many businesses and corporations run their networks on a cloud environment, which has its advantages and disadvantages. The advantage of a cloud network is a centralized place for data to be stored and a way to collect data. However, according to a study from the *Sensors* journal (Kelly et al, *Sensors* 2021, 21(7), 2433) describes the struggles of cloud computing, more specifically that “Cloud providers’ IP address subnets are public knowledge which opens channels for attackers to deploy mass scanners to automate attacks and take advantage of poorly configured services and protocols that are deployed in cloud instances.” (Kelly et al 2021.) Since Cybersecurity is an evolving field, regarding the security protocols and the different attacks, it is difficult to maintain a secure platform if companies are not fully aware of the developments in Cybersecurity. This is why implementing a Honeypot can be useful in the collection of data in regard to attacks. According to the same *Sensors* article, “By collecting data such as threat types, session duration, login credentials, and Common Vulnerability exposure numbers exploited, it allows security experts to advise organizations how they can improve their security posture” (Kelly et al 2021.) The data found from these Honeypots or HoneyNet systems can be used to discover what weaknesses their networks have, and collect this data without the risk of losing the real data within the company. This is due to the fact most Honeypot or HoneyNet systems are implemented in a controlled environment, and left vulnerable to these attacks.

In the *Sensors* Journal particular study, they tested 3 different Honeypot types including Dionaea, Glutton, and Cowrie. (Kelly et al, *Sensors* 2021, 21(7), 2433) Dionaea is designed to exploit a vulnerable windows environment, Glutton is a honeypot that specializes in capturing traffic logs and analyzing the traffic sent between the attacker and honeypot, and finally the Cowrie honeypot is designed to monitor brute force attacks. *Sensors’* research was able to show the origin country of the attacks, the volume of attacks, and how the attacks were carried out. For

example, the Cowrie attack showed that “attackers used automation as well as default credentials and passwords found in a dictionary in order to bypass authentication” (Kelly et al 2021.) In contrast to the Sensors journal article, another paper published by the International Association for Computer Information Systems implemented an SIEM database. The key difference between the two papers, is that the IACIS paper specifically used a University-hosted IP address in order to analyze threats in real time. (Holbel et al. 2024). The Security Information and Event Management system implemented was built on Ubuntu and Elastic Stack which “allowed for capturing and analyzing attacker behavior, providing valuable insights to the tactics, techniques and procedure employed by threat actors.” (Holbel et al. 2024) The team linked to the IACIS paper was able to use Elastic Stack to export logs and lock imported Binary files in a specified directory. (Holbel et al. 2024) The different implementation techniques can also benefit data collection analysis. Based on the research done above, it is inconclusive if one method worked better than the other, but both options highlight the different techniques that can be used to implement Honeypots or HoneyNet systems onto a network.

Although data collection and analysis are vital to the Honeypot’s purpose, it is vital to important privacy violations. This is due to the fact that there are several state and federal laws regarding data privacy against collection or use. Therefore, the concern of data privacy is a valid concern with some. In a paper published by HAL open science, “Collecting data raises a legal issue for honeynets as they could store attackers’ data as attackers may upload data into a honeypot such as payloads or documents” (Durand et al 2024.) The storage of the data collected can be challenging, as it varies by state or country. This is why properly storing the data is of the utmost importance. Recommended solutions to this issue would be only collecting the source IP of the attack and the methodology, especially for the Capstone project. This is because although

Honeypots can pose data integrity issues, collecting base level data that is collected by sites regardless would not necessarily pose any risk.

Honeynet Deception Strategies

Protecting the confidential information of a company, school or government is of the utmost importance. Using Honeypots, we are able to provide another layer of protection to the confidential information that we do not want other people accessing. As honeypots can not prevent the information from being stolen, it can help as a logging method to cybersecurity researchers. Deception is the leading term of honeypots, where a honeypot can have misinformation scattered throughout to allow the attacker to bring the information back to set back their team.

Psychology takes a large part into the deception of attackers. Drawing attention to bright colors and names that will grab their attention that may even have fake information; and a Honey file does just that. Naming a file called passwords and placing passwords to lab websites that are constantly being monitored by Cyber intelligence analysis (Kanellopoulos & Ioannidis, 2024, p. 5). Monitoring these logs allows researchers to strategically place honey pots, and honey files that will make up the honeynet. In the article *Leveraging competitive intelligence in offensive cyber counterintelligence: An operational approach for the shipping industry*, Kanellopoulos & Ioannidis talk about cyberwarfare in the shipping industry where no document or confidential piece of information is completely secure. Therefore leaking information for deception can draw attackers away from the true information. As the University of New Haven is not dealing with cyberwarfare, the university still has to deal with threat actors everyday where some still overlap

with the shipping industry and other areas of work. The University's cyberthreats can range from ransomware attacks to phishing schemes and supply chain breaches" (Kanellopoulos & Ioannidis, 2024, p. 6). Supply chain breaches and phishing schemes help get into the network. Having a threat actor that successfully conducts one of these threat vectors can be extremely costly and can lead to leaked data, and worse, a ransomware attack. Giving false information within your database to try to deceive the attacker is a protection method that has to be implemented into any honeynet.

Continuously, (Javadpour & others 2023) dedicated years of research to deception tactics that have been proved to work based on their own experiments. The team overlapped the ideas of where attacks can come from, but countermeasures to those attacks create different problems of protecting the confidential information. Honeypots and nets can plug in the holes that create problems, where we can be alerted when an intruder has been identified in the system. After the identification from our code we can place misinformation throughout to draw the attackers away from the true information. Javadpour recommended putting in a section that is heavily password protected that is not protecting anything, to draw the attention and time of the attacker to the fake section of the network infrastructure. Psychologically, people tend to naturally want a challenge, and when something appears to be a challenge, they will jump on it and try to solve the issue (Cohen, 2022, p 7). Give the attacker something to play with so the security team has plenty of time to kick them out, where that is one of the biggest purposes of a honeypot. The honeypot can go as far as giving the team a honeyfile that will draw the attacker even further away from the source that the attacker is trying to reach.

Legal and Ethical Considerations

When you use honeynets, several legal and ethical questions come up. Because honeynets deliberately attract hackers and collect information about them, you need to think carefully about the laws and rules that apply, plus what is fair and right.

From a legal point of view, one major thing is ownership. You must run a honeynet only on computers or networks you own or that you have clear permission to use. Putting a honeynet on someone else's network without their permission can break laws against unauthorized access. This is important because a honeynet looks like a vulnerable system that invites hackers. If you set one up on a network you don't own or don't have permission to use, you could break laws like the Computer Fraud and Abuse Act in the U.S. (Jarrett et al., n.d.).

Another legal concern relates to data privacy. Honeynets collect a lot of information about visitors to the fake network. This includes IP addresses, connection times, and possibly data transferred by attackers. Some of this data might be personal or sensitive, depending on the situation. If you operate a honeynet in places covered by laws like the EU's GDPR, you must handle this data properly, protect it, and respect privacy rights. Ignoring this can lead to fines or legal actions.

Entrapment is another topic. It usually applies to police, not private companies. But it's still important to avoid setting "traps" that push someone into doing something illegal. A 2024 paper says that honeynets are usually okay legally if they don't trick people, if they just sit there and wait, watching what comes in (Raghul et al., 2024). If you try to fake conversations, fake files, or fake accounts to pull someone in deeper, that could cause legal problems later, especially if you plan to use the data in court.

A big legal risk is if attackers use your honeynet to attack others. For example, if a hacker breaks into your honeynet and then uses your resources to attack a third party, you might get blamed. This means you need strong controls, like firewalls and monitoring, to keep the honeynet from being a launchpad for further attacks. Ignoring this risk can cause real legal trouble.

Ethics are also important. There is debate about whether organizations should tell users or the public they have honeynets running. Some think it's better to be transparent, while others think keeping honeynets secret helps catch more attackers. Most experts agree that honeynets should never be used to harm others beyond catching evidence. You should also avoid interfering too much with attackers. The goal is to observe and learn, not to fight back or cause damage. A 2019 paper explains that in many places, collecting attacker data from outside systems, without permission, can break the law even if the target is doing something wrong (Vetterl et al., 2019).

Maintaining the integrity of data collected is an ethical requirement. Do not alter or hide attacker data, because that can harm investigations or legal processes. Your honeynet must store information accurately and safely. Also, make sure the honeynet setup cannot accidentally harm innocent third parties, such as customers, partners, or other networks.

The Honeynet Project provides guidelines emphasizing clear policies, legal compliance, and risk management. They suggest a cautious, careful approach that balances security research with respect for laws and ethics (Honeynet Project, 2025). Running a honeynet responsibly means knowing what laws apply, protecting privacy, avoiding entrapment issues, controlling risks, and handling data properly.

In summary, honeynets can be effective tools for cybersecurity but come with legal and ethical responsibilities. If you own your systems, follow privacy laws, do not encourage illegal

actions, and prevent your honeynet from causing harm to others, you can run it responsibly.

Applying these principles lets you use honeynets safely and fairly to improve security knowledge and defenses.

Chapter III

METHODOLOGY

Introduction

The OWASP Honeynet Project implemented a honeynet which consisted of utilizing Virtual Machines, the CIT Network, PfSense, VSwitch, Command Line, Kibana, ELK Stack, and the Kali Linux Machines that were used to deploy and implement the Honeynet. The honeynet architecture, deployment, and implementation of the OWASP Honeynet project was done through vast research, trial and error, and continuous testing of the processes.

Design of the Project

The OWASP Honeynet Project has a network architecture that consists of a few different elements. The first layer of the OWASP honeynet experiment is the CIT Network. The Connecticut Institute of Technology Network acts as a barrier to the University's private network in order for our experiment to run on the proper channels and prevent any real-time attacks from taking place. Our architecture begins with the CIT Network. The CIT Network switch works as a firewall to protect data, which is connected to an isolated server. In the network architecture, there is also a PfSense virtual firewall that is responsible for filtering traffic and ensuring that no malicious activities are taking place. This firewall was placed between the VLAN segments and CIT Network firewall, and enabled the team to be able to see all incoming and outgoing traffic.

Beyond the firewall, there are 4 Virtual Machines. These were placed behind the protection of the CIT network, which allowed for the team to test them in a safe and controlled environment. Each of these virtual machines were home to their own honeypot, which targeted a different service that the university could provide. The honeypot that was implemented, was a database honeypot that simulated what sorts of attacks could take place on a database, and what security measures could be taken in order to prevent these attacks from happening.

The specific honeypot that was implemented was the Elasticpot Honeypot which emulated the behavior of an Elasticsearch database. This honeypot was examined to help the betterment of any existing database servers that exist in order to better the security infrastructure of the university. This honeypot was found through GitHub and by following the command line instructions was deployed. In order to receive attack data, the team had an opened command line interface to see the network traffic coming through the honeypot. Any attack data was sent to a syslog file in order for the team to compile results of attacks or attempts to access the Elasticpot server.

Lastly elasticpot is listening on port 9201, where the attacker can connect to the database to exploit any vulnerabilities that they may find. But all of this information is being followed as the honeypot elasticpot is logging their every movement. The team set up the ELK stack in which elasticsearch listening on port 9200 will take the syslog file that the elasticpot is writing to and allow logstash to feed the file into kibana. This now allows the team to visualize the logs and understand in an in depth analysis of what the attack is exploiting and how fast they are doing it.

Tools and Techniques Used

In our project we used a variety of tools. The tools that we used and their applications are listed below.

VSphere/VMWare: VSphere and VMWare were used to manage and operate the virtual machines. This allowed the team to run simulations and tests on Virtual Machines. It also allowed the team to organize their Virtual Machines and firewalls in one compact place.

PfSense: A PfSense Firewall was used to manage and filter traffic in order to allow traffic through but to also disallow malicious transactions.

Kali Linux: Kali Linux was the Disk Image/Operating System utilized in the testing of these honeypots. Kali Linux was incorporated in the simulation of attacks, deploying honeypots, and occasionally gathering web resources.

ELK Stack (Elasticsearch, Logstash, and Kibana): Used for log analysis, visualization, and monitoring of the honeypot activity. Kibana was used to display visual graphs and logs of the data.

Github: Used as a resource to locate Honeypots.

Chapter IV

DISCUSSION AND RESULTS

Introduction

This chapter provides results and analysis that was obtained through the OWASP Honeynet Experiment that was conducted. Through deploying the Elasticpot honeypot, the data collected reflects how the objectives of the project were met, what data was collected, and other key findings throughout the project.

Results and Discussion

The OWASP HoneyNet Experiment was done through utilizing the CIT production network which acted as a barricade to the outside world in order to protect the University from any real threats. Along with this, the experiment had utilized a PfSense firewall between VLAN segments and CIT firewall to help with the monitoring and filtering of traffic. There were 4 virtual machines that were deployed to house 4 different honeypots. Although only one honeypot was successful, the project was able to examine real-world threats behind a safe and controlled environment. The ElasticPot honeypot was deployed and allowed for data collection.

The ultimate goal of this project was to implement a honeypot and track attack patterns. While the team was unsuccessful in garnering attention from the outside world, the team was able to launch attacks from a local machine in order to test that the honeypot was able to track that data, and able to detect the attempts of entry or breach. The successful monitoring of traffic on the honeypot was met, as the team was able to examine line by line command line code of what the attacks were doing and where they originated from. This was also visualized within Kibana in a graphical and log format. The team was able to continue onto the next goal which was to study attack patterns and methodology to understand how attackers move through systems as the team was able to clearly see what was being done. For example, one attack showed the attempt of changing user information and passwords, and what time this attempt of attack happened. Finally, the team was able to determine a comprehensive list of recommendations for the University to take to improve the network including isolating systems that exhibit critical levels of breach, deploying firewalls and ensuring that they are up to date, and potentially having a honeypot of their own in order to study these attack patterns for themselves, as honeypots are a valuable prevention tool in order to keep networks safe.

Chapter V

CONCLUSION AND FUTURE WORK

Conclusion

The OWASP HoneyNet project aimed to help the security make security infrastructure changes in order to better protect the University of New Haven, and how the constant improvement of network services can improve the overall safety of information. Through the deployment of the honeypot through the Virtual Machines behind the protection of the CIT network and PfSense firewall, the project was able to show the attempt of attacks, while analyzing the real threats that can arise throughout the world in any circumstance.

Through deploying the 4 Virtual Machines, PfSense Firewall, and ELK Stack, the data collection shows that there is a need for security changes to be made regularly and made in accordance with current cybersecurity threats. Since security threats and cyber attacks are evolving everyday, it is important to have security measures and protocols in place in order to properly address them. By having raw data to display the attempt of attacks, while being able to visually see what attacks are being deployed, it gives those in positions of power an idea of what is currently happening in today's world. Honeypots are a preventative measure that should be taken by big corporations, businesses, and universities in order to collect data on who is trying to access their platforms and how the attackers are moving through the systems.

By using the specific data collected, the team is able to make a comprehensive list of security changes that the university can incorporate in keeping data safe. Through the attacks launched on the honeypot, the data shows that at any time, the right malicious user can gain

access to sensitive data if the networks are not properly secured. For example, in one of the attacks the users can see that attackers are attempting to change user information and gain access to password protected information. In a real-world scenario, this could include financial information, personal identifying information, and other sensitive data that if leaked, could cause detrimental effects to the University. By consistently updating firewalls and their rules, isolating systems that are showing significant levels of breach, and implementing their own honeypots, the University could get a better sense of what threats are happening in real-time, and what services may require more attention than others at any given moment. Through this project, honeypots are seen as a valid preventative tool for future use in order to continue to keep networks safe and secure from attacks.

Future work

To continue this work, the experiment could expand to involve multiple honeypots to create a honeynet in order to test the different vulnerabilities that other services could have. For example, implementing a DNS, SSH, or HTTP honeypot in order to examine more service specific threats would allow for the team to see what vulnerabilities may lie within a system. Expanding this project would also include gathering raw attack data from an attack that was not internally done. Our attack data were as a result of attacks done by the team. However, expanding this would include having attacks from other parts of the world including Russia and China. Finally, expanding this project would also giving the research conducted to businesses, corporations, and even the government in order to provide security recommendations to areas that need it the most.

The results of this project can be used for the University of New Haven to expand upon their security infrastructure and examine parts of the network that may need improvement. This

can help strengthen the security of student, faculty, and stakeholder information that may be at risk of being corrupted. The results of this project should also trigger the University to have some preventative measures in place, which may include a honeypot/honeynet of their own. This could help the University expand upon their current infrastructure and help protect highly sensitive data.

Appendix

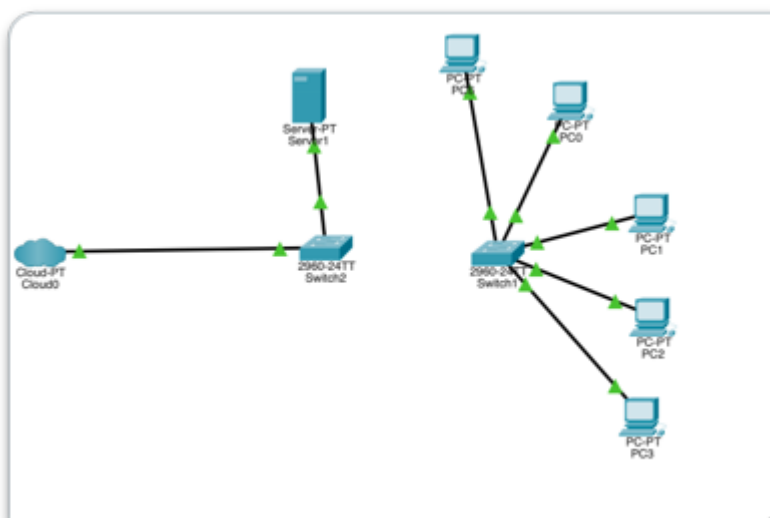


Figure 1: Network Architecture

The network architecture of our project as described in the Methodology section of the paper.

Start
View
Download
Clear Captures

Packet Capture Output: /tmp/packetcapture-vmx1-20250429143918.pcap

```

14:39:20.822060 IP 23.39.47.18.80 > 192.168.1.100.36438: tcp 0
14:39:21.911439 IP 192.168.1.102.40910 > 192.168.1.1.443: tcp 85
14:39:21.911453 IP 192.168.1.1.443 > 192.168.1.102.40910: tcp 0
14:39:21.911497 IP 192.168.1.102.40910 > 192.168.1.1.443: tcp 198
14:39:21.911500 IP 192.168.1.1.443 > 192.168.1.102.40910: tcp 0
14:39:21.911611 IP 192.168.1.1.443 > 192.168.1.102.40910: tcp 35
14:39:21.915252 IP 192.168.1.1.443 > 192.168.1.102.40910: tcp 856
14:39:21.915397 IP 192.168.1.102.40910 > 192.168.1.1.443: tcp 0
14:39:23.212963 IP 192.168.1.102.40032 > 34.95.113.255.443: tcp 0
14:39:23.222964 IP 34.95.113.255.443 > 192.168.1.102.40032: tcp 0
14:39:23.223130 IP 192.168.1.102.40032 > 34.95.113.255.443: tcp 0
14:39:23.223456 IP 192.168.1.102.40032 > 34.95.113.255.443: tcp 680
14:39:23.234107 IP 34.95.113.255.443 > 192.168.1.102.40032: tcp 0
14:39:23.244129 IP 34.95.113.255.443 > 192.168.1.102.40032: tcp 219
14:39:23.244243 IP 192.168.1.102.40032 > 34.95.113.255.443: tcp 0
14:39:23.244623 IP 192.168.1.102.40032 > 34.95.113.255.443: tcp 360
14:39:23.260072 IP 34.95.113.255.443 > 192.168.1.102.40032: tcp 0
14:39:23.346879 IP 34.95.113.255.443 > 192.168.1.102.40032: tcp 935
14:39:23.380097 IP 192.168.1.102.40032 > 34.95.113.255.443: tcp 0
14:39:23.768078 IP 0.0.0.0.0.68 > 255.255.255.255.67: udp, length 317

```

Figure 2: Firewall Packet Capture

```

The IPv4 LAN address has been set to 192.168.1.101/24
You can now access the webConfigurator by opening the following URL in your web
browser:
    http://192.168.1.101/

Press <ENTER> to continue.
Uptime Virtual Machine - Netgate Device ID: 792cc8a7c8bcee0b186b

*** Welcome to pfSense 2.7.2-RELEASE (amd64) on pfSense ***

LAN (wan)      -> vxo0      -> v4/DHCP4: 192.168.18.107/24
LAN (lan)      -> vxo1      -> v4: 192.168.1.101/24

0) Logout (SSH only)          9) pfTop
1) Assign Interfaces         10) Filter Logs
2) Set interface(s) IP address 11) Restart webConfigurator
3) Reset webConfigurator password 12) PHP shell + pfSense tools
4) Reset to factory defaults  13) Update from console
5) Reboot system             14) Enable Secure Shell (ssh)
6) Halt system               15) Restore recent configuration
7) Ping host                 16) Restart PHP-FPM
8) Shell

Enter an option:

```

Figure 3: Continued Firewall Set Up

In figures 2 and 3, the set up of the PfSense firewall is displayed as described in the Methodology section in the paper. The PfSense firewall acts as a first line of defense when detecting attacks, being located between the VLAN segment and the CIT Network Firewall. This allows the team to see all incoming and outgoing traffic.

```

capstonevm1@capstonevm1-virtual-machine: ~/elasticpot
capstonevm1@capstonevm1-virtual-machine:~$ cd elasticpot
capstonevm1@capstonevm1-virtual-machine:~/elasticpot$ sudo python3 elasticpot.py
[sudo] password for capstonevm1:
[2025-04-28 15:47:28.706609Z] Log opened.
[2025-04-28 15:47:28.706718Z] Elasticsearch Honeypot by Vesselin Bontchev
[2025-04-28 15:47:28.706749Z] Loading the plugins...
[2025-04-28 15:47:28.706825Z] Listening on port 9201.
[2025-04-28 15:47:28.706987Z] Site starting on 9201
[2025-04-28 15:47:28.707042Z] Starting factory <twisted.web.server.Site object at 0x77f35c0d1ae0>

```

Figure 4: Elasticpot Set Up

In figure 4, the set up of the Elasticpot honeypot is displayed as detailed in the Methodology section of the paper

```

Shell No. 1
File Actions Edit View Help
~/bin/bash

HONEYPOT_IP="192.168.1.102"
PORT="9201"
BASE_URL="http://$HONEYPOT_IP:$PORT"

echo "[*] Starting simulated attack on Elasticpot at $BASE_URL ..."

# 1. Search everything
echo "[*] Searching all documents ..."
curl -s "$BASE_URL/_search?q=*:*pretty"

# 2. Search for sensitive fields
echo "[*] Searching for sensitive fields ..."
curl -s "$BASE_URL/_search?q=password:*S280K52p*password:*S280K52bus*
:oon52email:*pretty"

# 3. Attempt bulk document extraction
echo "[*] Attempting full index dump ..."
curl -s "$BASE_URL/_logstash-*/_search?size=100000q=*:*pretty"

# 4. Attempt to insert malicious doc
echo "[*] Attempting to insert fake document ..."
curl -s -X POST "$BASE_URL/_index/_doc/" -H "Content-Type: application/json" -d '{
  "username": "admin",
  "password": "hacked",
  "note": "this is a test attack"
}'

# 5. Attempt to delete index
echo "[*] Attempting to delete index ..."
curl -s -X DELETE "$BASE_URL/_logstash-2025.04.12"

# 6. Ransomware-style message
echo "[*] Sending fake ransomware message ..."
curl -s -X POST "$BASE_URL/_message/_doc/" -H "Content-Type: application/json" -d '{
  "msg": "Your data has been encrypted. Send 1 BTC to wallet address or all data will

```

Figure 5: Script attack on Kali Linux machine

The attacks run on a local machine specified in the methodology and results section of the paper seen in figure 5.

```

capstonevm1@capstonevm1-virtual-machine: ~/elasticpot
capstonevm1@capstonevm1-virtual-machine:~/elasticpot$ sudo python3 elasticpot.py
[sudo] password for capstonevm1:
[2025-04-15 17:53:46.626539Z] Log opened.
[2025-04-15 17:53:46.626694Z] Elasticsearch Honeypot by Vesselin Bontchev
[2025-04-15 17:53:46.626694Z] Loading the plugins...
[2025-04-15 17:53:46.626769Z] Listening on port 9201.
[2025-04-15 17:53:46.626935Z] Site starting on 9201
[2025-04-15 17:53:46.626987Z] Starting Factory <twisted.web.server.Site object at 0x7913843b9b>
[2025-04-15 23:31:56.819802Z] [INFO] (192.168.1.101:59192): GET: /_search?q=*:*pretty
[2025-04-15 23:31:56.866454Z] [INFO] (192.168.1.101:59208): GET: /_search?q=password:* OR pas
[2025-04-15 23:31:56.872975Z] [INFO] (192.168.1.101:59218): GET: /logstash-*/*_search?size=1000
[2025-04-15 23:31:56.882616Z] [INFO] (192.168.1.101:59228): POST: /_index/_doc/
[2025-04-15 23:31:56.882690Z] [INFO] (192.168.1.101:59228): POST body:
{
  "username": "admin",
  "password": "hacked",
  "note": "this is a test attack"
}
[2025-04-15 23:31:56.897269Z] [INFO] (192.168.1.101:59246): POST: /_message/_doc/
[2025-04-15 23:31:56.897344Z] [INFO] (192.168.1.101:59246): POST body:
{
  "msg": "Your data has been encrypted. Send 1 BTC to wallet address or all data will

```

Figure 6: Results of Attack

Line by line results of the attack run in Figure 5, as detailed in the methodology and results section of the paper.



Figure 7: Graphical Display of Results



Figure 8: Log Display of Results

Figures 7 and 8 detail the results of the attacks being recorded and logged as detailed in the results and methodology section. Figure 7 shows the amount of data being attacked versus the time it occurred, while Figure 8 shows more detailed log information of each instance.

REFERENCES

A Distributed Cloud Honeypot Architecture. (2021, July 1). IEEE Conference Publication | IEEE

[Xplore. https://ieeexplore.ieee.org/abstract/document/9529788](https://ieeexplore.ieee.org/abstract/document/9529788)

Cohen, Fred. "The Use of Deception Techniques: Honeypots and Decoys." *The Use of*

Deception Techniques: Honeypots and Decoys, 2023,

all.net/journal/deception/Deception_Techniques_.pdf.

Holbel, R., Yerby, J., & Smith, W. (2024). Utilizing virtualized honeypots for threat hunting, malware analysis, and reporting. *International Association for Computer Information Systems*, 25(1), 265-278.

Jarrett, H. M., Bailie, M. W., & OLE (Directors). (n.d.). Prosecuting computer crimes. In OLE & S. Eltringham (Eds.), *OLE*.

Javadpour, A. J., Ja'fari, F. J., Taleb, T. T., Shojafar, M. S., & Benzaid, C. B. (2024). A comprehensive survey on cyber deception techniques to improve honeypot performance. *ScienceDirect*, 140, 103792.

<https://www.sciencedirect.com/science/article/pii/S0167404824000932>

Kaur, J., Singh, G., Singh, M., SMCA, Thapar University, Patiala -147004, India, & CSE, Ramgharia College, Phagwara, India. (2012). Design & Implementation of Linux based Network Forensic System using HoneyNet. *International Journal of Advanced Research in Computer Engineering & Technology*, 504

Kanellopoulos, Anastasios-Nikolaos, and Antonios Ioannidis. "Leveraging Competitive Intelligence in Offensive Cyber Counterintelligence: An Operational Approach for the Shipping Industry." *Security and Defence Quarterly*, War Studies University, Poland, 25 Oct. 2024, <https://securityanddefence.pl/Leveraging-competitive-intelligence-in-offensive-cyber-counterintelligence-An-operational,192342,0,2.html>

Kelly, C., Pitropakis, N., Mylonas, A., McKeown, S., & Buchanan, W. J. (2021). A Comparative Analysis of Honeypots on Different Cloud Platforms. *Sensors*, 21(7).

Lackner, P. (2021). How to Mock a Bear: Honeypot, HoneyNet, HoneyWall & HoneyToken: A Survey. *How to Mock a Bear: Honeypot, HoneyNet, HoneyWall & HoneyToken: A Survey*, 181–188. <https://doi.org/10.5220/0010400001810188>

Mathis Durand, Yvon Kermarrec, Marc-Oliver Pahl. Real-world detection and mitigation of AI-based cyberattacks and defence mechanisms. 2024. fhal-04603879

Omar, A. H. E., Soubra, H., Moulla, D. K., & Abran, A. (2024). An innovative honeypot architecture for detecting and mitigating hardware trojans in IoT devices. *IoT*, 5(4), 730–755. <https://doi.org/10.3390/iot5040033>

“OWASP/Python-Honeypot: OWASP Honeypot, Automated Deception Framework.”

GitHub, Zedresearch, 24 Oct. 2021, github.com/OWASP/Python-Honeypot.

Provos, N. & Google, Inc. (2004). A virtual honeypot framework. In *Proceedings of the 13th USENIX Security Symposium*[Journal-article].
https://www.usenix.org/legacy/publications/library/proceedings/sec04/tech/full_papers/provos/provos.pdf

Raghul, S. A., Gayathri, G., Bhatt, R., & Kumar, K. a. V. (2024). Enhancing cybersecurity resilience: integrating IDS with advanced honeypot environments for proactive threat detection. *Enhancing Cybersecurity Resilience: Integrating IDS With Advanced Honeypot Environments for Proactive Threat Detection*, 1363–1368.
<https://doi.org/10.1109/icaaic60222.2024.10575865>

Spitzner, L. (2002). *Honeypots: Tracking Hackers*. <http://cds.cern.ch/record/1505952>

The Honeynet project. (2012, March 28).

<https://www.honeynet.org/2012/03/28/thoughts-on-the-microsoft-s-operation-b71-zeus-botnet-civil-legal-action/>

Vetterl, A., Clayton, R., & Walden, I. (2019). Counting Outdated Honeypots: Legal and useful. *Counting Outdated Honeypots: Legal and Useful*, 224–229.
<https://doi.org/10.1109/spw.2019.00049>

“What Is a Honeypot: Honeynets, Spam Traps & More.” *Imperva*, 20 Dec. 2023,

www.imperva.com/learn/application-security/honeypot-honeynet/.